Mise en place de l'application

1. Description de l'existant

L'application se compose de 3 projets :

Le projet **Métier** de type 'bibliothèque de classe' comporte l'ensemble des classes métiers et la classe Session permettant de conserver toutes les données de la session en cours
Les classes utilisent des propriétés automatiques

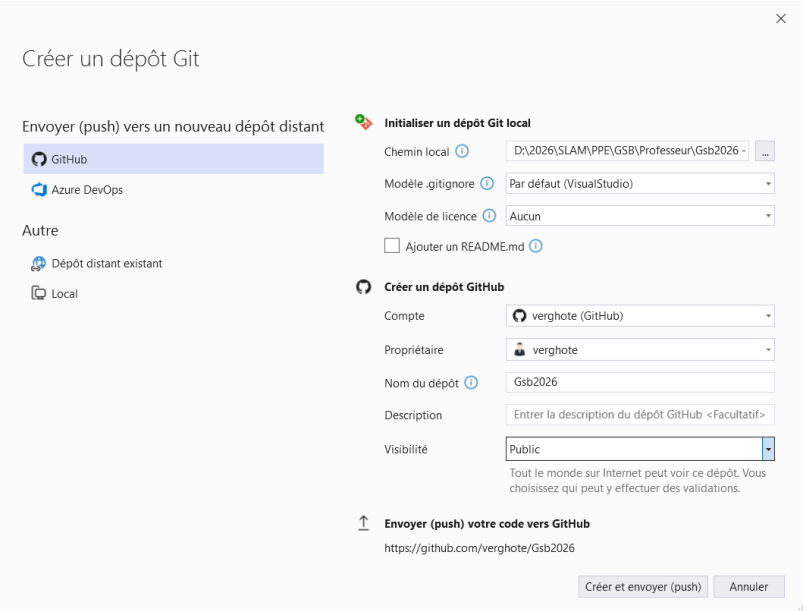
Le projet **Interface** de type WinForm contient les formulaires du projet. Actuellement on y trouve :
Le formulaire de connexion qui sera lancé au démarrage sous la forme d'une fenêtre modale
Le formulaire FrmBase qui sera hérité par tous les formulaires de l'application. Il contient les éléments communs : entête (menu), pied, traitements communs et surtout il transmet par le principe d'injection de dépendance l'objet Session contenant l'ensemble des données associées au visiteur connecté
Le formulaire FrmMenu qui correspond au formulaire principal de l'application. Si ce formulaire est fermé l'application se ferme automatiquement.
Le formulaire FrmVisiteAjout à compléter permettant d'ajouter une nouvelle visite.

Le Projet **Donnee** de type bibliothèque de classe contenant la classe Passerelle chargée de récupérer les données depuis la base de données et de réaliser les opérations de mise à jour.

2. Mise en place d'un dépôt sur GitHub

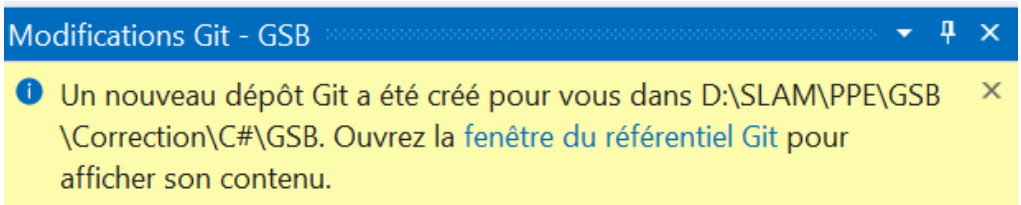
Visual studio permet de générer à la fois le dépôt local et le dépôt distant sur GitHub et de lancer le premier 'push'
En bas de la fenêtre on trouve :

En cliquant sur Ajouter la fenêtre suivante s'ouvre :



1. Créer le dépôt Git local et le dépôt distant sur Github
2. Vérifier la création du dépôt sur votre compte Github

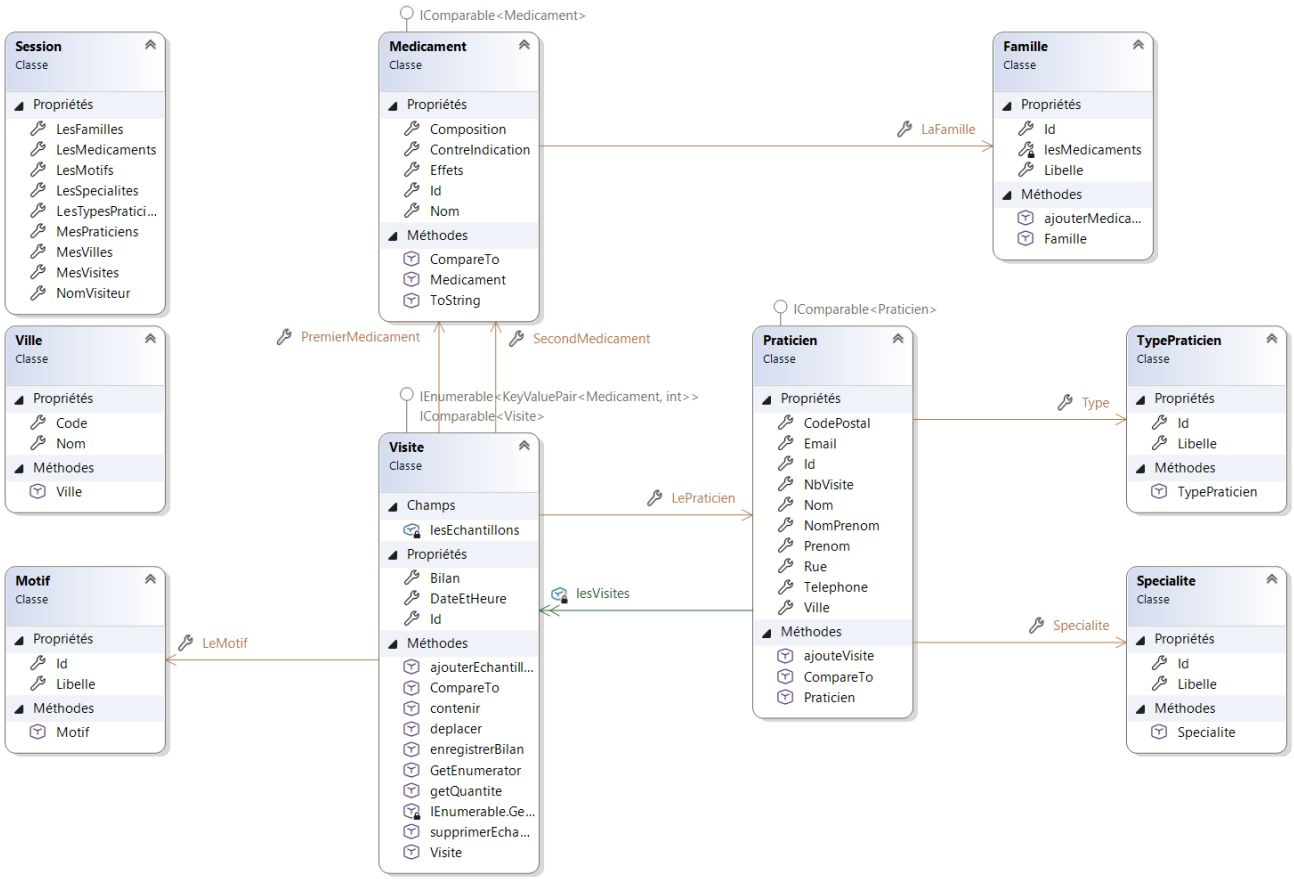
Un petit mot sur les licences :
Développement > Généralités > les modèles de licences.pdf et les licences créative commons.pdf



Autre moyen d'accès : Menu Git – Créer un dépôt Git

3. Réalisation des diagrammes de classe

3.	Ajouter dans le projet Metier, un nouvel élément de type ' Diagramme de classes'
4.	Faire glisser chaque classe, dans ce diagramme et modifier le format des membres afin d'obtenir le schéma ci-dessous.



La classe Visite comporte un dictionnaire pour stocker les médicaments fournis en échantillon, la clé est le médicament et la valeur représente la quantité fournie

4.

5. La classe Visite

La classe Visite hérite de l'interface Icomparable<visite> afin de définir le critère de tri sur la date et heure

La classe Visite hérite de l'interface IEnumerable<KeyValuePair<Medicament, int>> afin de permettre de parcourir le dictionnaire lesEchantillons sur ses couples "clé-valeur"

La relation bidirectionnelle entre visite et Praticien doit être prise en charge dans le constructeur

5. Compléter le constructeur de la classe Visite

L'ajout d'un échantillon vient augmenter la quantité si le médicament est déjà dans le dictionnaire sinon il faut simplement ajouter cet échantillon dans le dictionnaire.

6. Compléter la méthode ajouterEchantillon

6. Les méthodes de la classe Passerelle utilisées lors de la connexion du visiteur

Elle assure toutes les interactions avec la base de données.
Son objectif principal est de charger et gérer toutes les données nécessaires à une session utilisateur.

_chaineConnexion : stocke la chaîne de connexion à la base de données qui est ici dynamique afin de s'adapter au visiteur.

ouvrirConnexion() : instancie et ouvre une connexion MySQL à partir de _chaineConnexion.

```
private static MySqlConnection ouvrirConnexion()  {
    MySqlConnection cnx = new MySqlConnection(_chaineConnexion);
    cnx.Open();
    return cnx;
 }
```

ouvrirSession(login, mdp, out message) : méthode principale pour authentifier un utilisateur.

Initialise la chaine de connexion
Vérifie les identifiants via MySQL.
En cas de succès, retourne un objet Session contenant toutes les données nécessaires.
En cas d'échec, renvoie un message d'erreur explicite.

A noter : Dans le cadre d'une application multi-utilisateur, la chaine de connexion devrait être conservé non pas dans la classe Passerelle mais dans la classe Session et toutes les méthodes de mise à jour devraient passer l'objet session en paramètre.

Chaque visiteur dispose de son propre compte utilisateur sur la base de données. Cette technique permet de s'assurer que le visiteur ne pourra jamais accéder aux données des autres visiteurs

S'ils les paramètres de connexion sont corrects les données concernant ce visiteur sont chargées dans en objet Session en appelant la méthode chargerDonneesSession(cnx)

Cette méthode récupère toutes les informations depuis la base de données et les organise dans des objets métiers :

Nom du visiteur : getNomVisiteur(cnx)
Motifs de visite : chargerLesMotifs(cnx)
Types praticien : chargerLesTypesPraticien(cnx)
Spécialités des praticiens : chargerLesSpecialites(cnx)
Familles de médicament ; chargerLesFamilles(cnx)
Villes associées au visiteur (chargerMesVilles)

Médicaments : chargerLesMedicaments(cnx, lesFamilles)
Praticiens : chargerMesPraticiens(cnx, lesTypes, lesSpecialites)
Visites : chargerMesVisites(cnx, lesMotifs, lesPraticiens, lesMedicaments)
Échantillons distribués lors des visites : chargerMesEchantillons(cnx, lesVisites, lesMedicaments)

7. Compléter la méthode getNomVisiteur de la classe passerelle

8. En observant la méthode chargerLesMotifs, compléter les méthodes chargerMesVilles

Pour générer les collections concernant les médicaments, les praticiens et les visites il est nécessaire de récupérer des objets provenant des autres collections

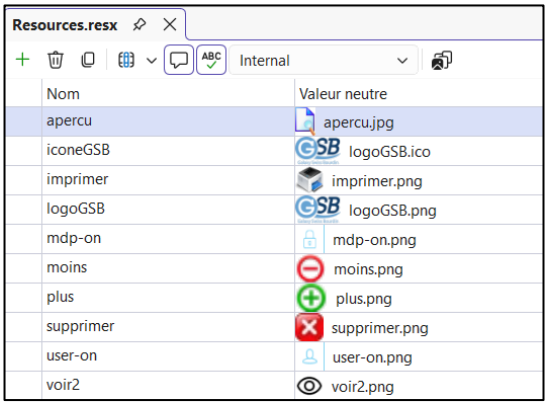
La recherche dans une collection peut être réalisée à l'aide de la méthode Find. Toutefois, cette méthode effectue un parcours séquentiel des éléments, ce qui peut devenir peu performant lorsque le volume de données augmente.

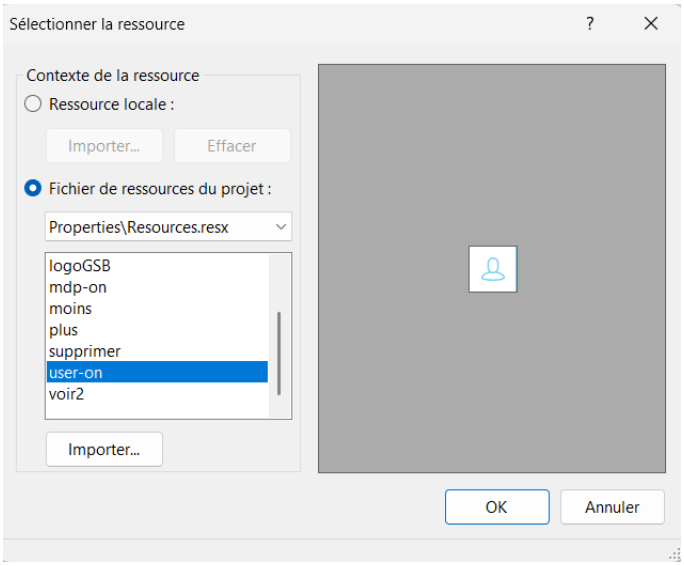
Il est donc préférable d'utiliser un dictionnaire, qui permet un accès direct aux éléments par clé. La recherche s'effectue alors en temps constant, indépendamment du nombre d'éléments contenus dans la collection.

9. En observant les méthodes chargerMesPraticiens et chargerMesVisites compléter les méthodes chargerLesMedicaments et chargerMesEchantillons

7. Le formulaire de connexion

Le formulaire comporte trois images. Toutes les images du l'application sont regroupées dans le dossier Ressources du projet Interface.
Pour Ajouter une image dans les ressources, il faut ouvrir le fichier Resources.resx du dossier Properties.





En étudiant le code répondre aux questions suivantes :

10. Comment remplir automatiquement le formulaire de connexion ?
11. Doit-on obligatoirement cliquer sur le bouton 'Se connecter' pour valider le formulaire ?
12. Comment a-t-on retirer les boutons réduire et agrandir de la fenêtre ?
13. Comment ce formulaire est-il ouvert au démarrage de l'application ?
14. Que se passe-t-il lorsque le visiteur valide ce formulaire ?

8. Le programme principal

Il lance le formulaire Menu une fois la connexion du visiteur réussie.

Un des problèmes classiques lorsqu'on utilise des Windows Form est que la fermeture du premier formulaire ouvert entraine forcément la fermeture de l'application.

Pour contourner ce problème dans notre application, le formulaire de connexion sera ouvert dans une fenêtre modale. L'ouverture sera déclenchée indirectement par le fichier Program.cs en appelant la méthode statique getSession de la classe FrmConnexion

En passant par l'appel de cette méthode, cela permet de récupérer l'objet Session associé au visiteur connecté

```
static void Main()  {
    try   {
        Session? session = FrmConnexion.getSession();
        if (session is not null)     {
            Application.Run(new FrmMenu(session));
        }
    } catch (OperationCanceledException)  {
            Application.Exit();
    }
```

Important à savoir : Les formulaires sont techniquement des instances de leur classe, il est donc possible de créer une instance sans forcément l'afficher immédiatement.

9. Le formulaire de base

La classe FrmBase est la classe de base pour tous les formulaires de l'application. Elle fait partie du namespace Interface et sert à centraliser la gestion commune aux différents écrans de l'application.

Il comporte un entête composé d'un menu de l'application ainsi que titre, un emplacement pour le corps et un pied de page affichant le nom du visiteur connecté.

Il contient un objet Session qui stocke les données de l'utilisateur connecté et les informations nécessaires à l'application.

Il permet la transmission de la session aux formulaires secondaires via injection de dépendance.

La méthode OuvrirFormulaire(Form frm) permet d'ouvrir un formulaire enfant et doit masquer ou fermer le formulaire parent selon le contexte. En effet la fermeture du premier formulaire ouvert par l'application entraine forcément la fermeture de l'application. Il faut donc fermer le formulaire (méthode Close) courant sauf s'il s'agit du formulaire menu (instance de la classe FrmMenu) et dans ce cas il faut masquer le formulaire (méthode Hide)

15. Compléter la méthode ouvrirFormulaire()

Un problème se pose lors de la fermeture d'un formulaire secondaire
L'application n'affiche plus aucun formulaire mais elle continue à tourner puisque le formulaire menu est caché en mémoire.

Comment arrêter correctement l'application à la fermeture d'un formulaire ?
On peut déclencher un traitement après la fermeture d'un formulaire en utilisant l'événement **FormClosed** qui se produit une fois le formulaire fermé

Si on peut connaitre le nombre de formulaires en mémoire après la fermeture on règle notre problème.

La règle est la suivante :

S'il reste un seul formulaire en mémoire (**Application.OpenForms.Count = 1**) il s'agit forcément du formulaire menu et on peut arrêter proprement l'application en exécutant la commande suivante : **Application.ExitThread();**

Sinon, il en reste forcément 2, le formulaire Menu et le formulaire qui vient d'être appelé et qui va s'ouvrir et dans ce cas il n'y a rien à faire, l'application ne va pas se fermer, elle va afficher le nouveau formulaire ouvert

16. Compléter la procédure événementielle FormClosed

Les événements des menus et boutons (ex : consulter, ajouter, modifier visites ou praticiens) créent et ouvrent les formulaires correspondants en transmettant la session.

Certaines options du menu doivent être activées uniquement si une condition est réalisée, à vous de les trouver

17. Compléter la méthode ParametrerComposant

10. Le formulaire Menu frmMenu

Le formulaire FrmMenu hérite comme tous les autres formulaires de l'application de la classe FrmBase

```
public partial class FrmMenu : FrmBase
```

Cet héritage permet d'avoir une mise en forme commune pour tous les formulaires (entête et pied de page et titre de la page)
Le formulaire Menu n'ayant pas de corps il doit simplement définir le titre de la page

```
private void FrmMenu_Load(object sender, EventArgs e)
    => lblTitre.Text = "Bienvenue sur l'application de gestion des visites";
```

11. Ajout d'un nouveau formulaire

18. Ajouter dans le projet Interface un Windows Form '**FrmVisiteAjout**'

Clic droit sur le projet > Ajouter
Nom : FrmVisiteAjout.cs
FrmVisiteajout doit hériter FrmBase pour disposer du menu et du pied commun à tous les formulaires

```
public partial class FrmVisiteAjout : FrmBase
```

En ouvrant FrmVisiteAjout.cs en mode designer on peut voir apparaitre le menu, le pied et le titre hérité de FrmBase

19. Décommenter la méthode ci-dessous dans FrmBase

```
private void programmerRendezVous_Click(object sender, EventArgs e)  {
        OuvrirFormulaire<FrmVisiteAjout>();
}
```

20. Lancer l'application

Une erreur est signalée : 'FrmVisiteAjout' ne contient pas de constructeur qui accepte des arguments 1

21. Modifier le constructeur de la classe FrmVisiteAjout et relancer l'application

12. Mise à jour des dépôts Git et Gitbub

22. Mettre à jour votre dépôt GitHub à partir de Visual Studio ou à l'aide des commandes git

Message du commit : **Etape 1 Mise en place de l'application**